

EC2 and Auto Scaling

Instance Types · Launch Templates · ASGs · Spot Instances · SSM · EBS · Image Builder

AWS Tutorial · ShopFlow

04-00

Chapter Overview — EC2 and Auto Scaling

EC2 (Elastic Compute Cloud) is AWS's virtual machine service. While ShopFlow runs containerized workloads on ECS Fargate (no direct EC2 management), understanding EC2 is fundamental: every container runs on top of an EC2 instance somewhere. Developers use EC2 for bastion hosts, build servers, and database migration tools. Auto Scaling ensures your application handles both quiet nights (2 servers) and Black Friday (40 servers) automatically.

■ EC2 in Plain English

EC2 is like renting computers in the cloud. You pick: how powerful (instance type like t3.medium = 2 vCPUs, 4 GB RAM), what operating system (Amazon Linux 2023, Ubuntu 22.04), and how long you need it (on-demand: pay by the hour; reserved: 1-3 year commitment for 40-60% discount). Unlike a physical computer, you can start a new EC2 instance in 60 seconds and terminate it when done — paying only for what you use.

Example: ShopFlow launches a t3.medium EC2 bastion host for database administration. Cost: \$0.0416/hr = ~\$30/month. During a migration project, the team launches 10 c6g.4xlarge instances (\$0.544/hr each) for 8 hours to process a data transformation — total cost: \$43.52. Then terminate them. No long-term commitment.

EC2 Concept	What It Is	ShopFlow Usage
Instance Type	Defines vCPUs, RAM, network bandwidth, and storage performance	Bastion host (\$0.0104/hr), c6g.4xlarge for migration
AMI	Amazon Machine Image: a snapshot of an OS with pre-installed software	Amazon Linux 2023 AMI: includes AWS CLI, SSM agent, CloudWatch agent
Key Pair	SSH key pair for accessing EC2 instances	Used for bastion host (though SSM Session Manager is preferred)
User Data	Shell script run on first boot to configure the instance	Install Java 21, configure CloudWatch agent, register with ECS
Instance Profile	IAM role attached to an EC2 instance (similar to ECS task role)	Bastion instance profile: allows SSM Session Manager and CloudWatch access
Placement Group	Controls how EC2 instances are placed physically	Cluster placement group: put all instances on same rack for low latency

04-01

EC2 Instance Types — Choosing the Right Size

AWS offers 500+ EC2 instance types organized into families. Picking the wrong type wastes money (oversized) or causes performance issues (undersized). The family tells you the use case; the size tells you how much. ShopFlow uses different instance types for different jobs.

Family	Use Case	Examples	ShopFlow Use
t3/t4g	General purpose, burstable CPU	t3.micro, t3.small, t3.medium, t3.large	Bastion host (t3.micro), dev environment (t3.medium)
m6g/m7g	General purpose, balanced CPU	m6g.large (2vCPU/8GB), m6g.xlarge (8vCPU/32GB)	Application container hosts (AWS manages these; y
c6g/c7g	Compute optimized — high CPU	c6g.4xlarge (40vCPU/256GB)	Batch data migration jobs, image resizing Lambda (Gr
r6g/r7g	Memory optimized — large RAM	r6g.2xlarge (8vCPU/64GB)	ElastiCache Redis hosts (AWS manages; large cache
i3/i4i	Storage optimized — high-speed NVMe	i3.xlarge (8vCPU/16GB/475GB NVMe)	Not used in ShopFlow (Aurora manages storage; El
p3/p4	GPU compute for ML training	p3.2xlarge (8vCPU/61GB/1x V100 GPU)	Future: product recommendation ML model training

04-02

Launching Your First EC2 Instance — Bastion Host

A bastion host (also called a jump server) is a small EC2 instance in a public subnet that provides secure access to resources in private subnets. ShopFlow uses a bastion host for database administration — DBAs connect to the bastion, then from the bastion connect to Aurora. However, modern practice is to use SSM Session Manager instead of SSH, which eliminates the need to expose port 22.

■ Launch ShopFlow Bastion Host
1. EC2 → Instances → Launch instances
2. Name: shopflow-bastion-host
3. AMI: Amazon Linux 2023 AMI (64-bit, x86) — free tier eligible
4. Instance type: t3.micro (1 vCPU, 1 GB RAM — sufficient for a bastion)
5. Key pair: Create new key pair → shopflow-bastion-key → .pem format → Download
6. Network settings: VPC: shopflow-vpc, Subnet: shopflow-subnet-public1 (a public subnet)
7. Auto-assign public IP: Enable (bastion needs a public IP to be reachable)
8. Firewall (Security Groups): Create security group: shopflow-bastion-sg
9. Inbound: SSH (22) from My IP (your current IP only — not 0.0.0.0/0)
10. Configure storage: 20 GB gp3 — leave defaults
11. Advanced details → IAM instance profile: create a new profile with AmazonSSMManagedInstanceCore policy
12. Click Launch instance

Launch an instance

Name

shopflow-bastion-host

■ Tag: Name

■ Application and OS Images (Amazon Machine Image)

AMI

■ Amazon Linux 2023 AMI (Free tier eligible) ■ Ubuntu Server 22.04 LTS ■ Windows Server 2022 ■ Browse more AMIs

■ Instance type

Instance type

t3.micro (1 vCPU | 1 GiB Memory | \$0.0104/hr) ■

■ Network settings

VPC

shopflow-vpc

Subnet

shopflow-subnet-public1-us-east-1a

Auto-assign public IP

■ Enable ■ Disable

■ Create or select a security group that allows SSH (port 22) from your IP only, not from 0.0.0.0/0

Launch instance

04-03

SSM Session Manager — No SSH Needed

AWS Systems Manager (SSM) Session Manager lets you open a terminal to any EC2 instance without SSH, without a bastion host, and without opening port 22 in any security group. The instance connects outbound to the SSM service. You connect via the AWS console or CLI. All sessions are logged to CloudWatch. ShopFlow uses Session Manager for all EC2 access.

■ Connect to EC2 Using SSM Session Manager (No SSH)

1. Ensure the EC2 instance has the AmazonSSMManagedInstanceCore IAM policy attached
2. Ensure the instance has SSM agent installed (Amazon Linux 2023 has it pre-installed)
3. EC2 → Instances → select shopflow-bastion-host → Connect
4. Choose: Session Manager tab (not SSH client tab)
5. Click Connect — a browser-based terminal opens in the console
6. You are now connected as ssm-user with sudo privileges
7. All commands typed are logged to CloudWatch Logs automatically
8. Session terminates automatically after 20 minutes of inactivity (configurable)

Access Method	Port 22 Required?	Key Pair Required?	Audit Logging	Recommended?
SSH directly to public IP	Yes — port 22 exposed to internet	Yes — must distribute .pem (if not a custom AMI)	Yes — all sessions logged to CloudWatch logs	No — exposed attack surface; key management; full audit trail
SSH via bastion host	Yes on bastion (port 22)	Yes — key pair for bastion	Yes — (unless custom AMI)	Acceptable but outdated; bastion cost
SSM Session Manager	No — zero open inbound ports	No — IAM authentication	Yes — all sessions logged to CloudWatch logs	Yes — recommended; full audit trail

04-04

EBS Volumes — Persistent Storage for EC2

Every EC2 instance has an EBS (Elastic Block Store) root volume — like the hard drive in your computer. EBS volumes persist independently of the EC2 instance: stop the instance, the data remains. EBS supports different volume types optimized for different workloads. ShopFlow uses gp3 volumes for all general-purpose storage.

Volume Type	Performance	Use Case	Cost
gp3 (General Purpose SSD)	3,000 IOPS baseline; up to 16,000 IOPS (independent of size)	Default for EC2 root volumes, applications, databases	\$0.08/GB/month (20K IOPS)
io2 Block Express (Provisioned IOPS SSD)	Up to 1,000,000 IOPS; sub-millisecond latency	High-performance databases requiring consistent high IOPS	\$0.125/GB/month (1M IOPS)
st1 (Throughput Optimized HDD)	Up to 500 MB/s throughput (sequential reads)	Data warehouses, log processing, large sequential reads	\$0.045/GB/month
sc1 (Cold HDD)	Up to 250 MB/s; lowest cost per GB	Infrequently accessed data, archive	\$0.015/GB/month

```
// CDK: EC2 instance with gp3 root volume
const bastion = new ec2.Instance(this, "Bastion", {
  instanceType: ec2.InstanceType.of(ec2.InstanceClass.T3, ec2.InstanceSize.MICRO),
  machineImage: ec2.MachineImage.latestAmazonLinux2023(),
  vpc,
  vpcSubnets: {subnetType: ec2.SubnetType.PUBLIC},
  securityGroup: bastionSg,
  blockDevices: [{
    deviceName: "/dev/xvda",
    volume: ec2.BlockDeviceVolume.ebs(20, {
      volumeType: ec2.EbsDeviceVolumeType.GP3,
      encrypted: true, // always encrypt EBS volumes
      deleteOnTermination: true,
    }),
  }],
  ssmSessionPermissions: true, // enables SSM Session Manager automatically
});
```

04-05

Auto Scaling Groups — Handle Any Traffic Level

An Auto Scaling Group (ASG) maintains a fleet of EC2 instances that automatically scales up when traffic increases and scales down when it drops. While ShopFlow uses ECS Fargate (AWS manages the EC2 fleet), understanding ASGs is critical because: ECS on EC2 uses ASGs, many teams use ASG-based architectures, and Auto Scaling concepts apply directly to ECS service scaling.

■ Create an Auto Scaling Group for a Web Tier

1. EC2 → Auto Scaling Groups → Create Auto Scaling group
2. Name: shopflow-web-asg
3. Launch template: Create launch template → same settings as the bastion (instance type, AMI, SG, IAM profile)
4. VPC: shopflow-vpc | Subnets: select all 3 private app subnets (one per AZ)
5. Load balancing: Attach to existing ALB target group: shopflow-web-tg
6. Health checks: EC2 + ELB (mark unhealthy if either EC2 or health check fails)
7. Group size: Minimum 3, Desired 3, Maximum 30
8. Scaling policies: Target tracking → Average CPU Utilization → 60%
9. Instance warmup: 300 seconds (let new instances warm up before counting in metrics)
10. Click Create Auto Scaling group

■ Auto Scaling Policies

Target tracking: set a target metric (CPU=60%) and ASG automatically adds/removes instances to maintain it. Scale-out when CPU > 60%, scale-in when it drops. Simple scaling: "add 2 instances when CPU > 80%" — manual threshold rules. Scheduled scaling: "add 10 instances at 8am Monday, remove them at 8pm Friday" — predictable traffic patterns. ShopFlow uses target tracking for day-to-day and scheduled scaling for Black Friday.

Example: Black Friday at ShopFlow: scheduled action at 8pm Thursday → set min=20, desired=20. Adds 17 instances before Black Friday midnight traffic hits. Target tracking then handles traffic spikes within the provisioned 20-40 range. At 9am Friday → scheduled action scales back to min=5.

04-06

Launch Templates — Define Your EC2 Configuration

A Launch Template defines the configuration for EC2 instances launched by an Auto Scaling Group: AMI, instance type, security group, IAM profile, EBS volumes, user data script. Launch Templates support versioning — make a change, create a new version, roll back if needed. This is the modern replacement for Launch Configurations.

■ aws > EC2 > Launch Templates > Create launch template

Create launch template

Launch template name

shopflow-web-lt

Version description

Amazon Linux 2023 + Java 21 + CloudWatch agent

■ Application and OS Images

AMI

ami-0c55b159cbfafa1f0 — Amazon Linux 2023 AMI (64-bit x86) ■

■ Instance type

Instance type

m6g.large (2 vCPU | 8 GiB | \$0.077/hr) — Graviton3; 40% better price/perf ■

■ Network settings

Security groups

shopflow-ecs-sg ■

■ Do NOT assign a public IP to application tier instances — they are in private subnets

■ Advanced details

IAM instance profile

shopflow-ec2-instance-profile ■

■ User data: paste the shell script that installs Java 21, registers with ECS cluster, and starts CloudWatch agent

Create launch template

04-07

EC2 Pricing — On-Demand vs Reserved vs Spot

EC2 pricing has four models. Choosing the right model for each workload is one of the highest-impact cost optimizations available. ShopFlow uses all four models for different parts of the system.

Pricing Model	Description	Discount vs On-Demand (baseline price)	ShopFlow Use Case
On-Demand	Pay by the hour or second; no commitment; can stop anytime	0% (baseline price)	Unpredictable workloads: migration jobs, dev
Reserved Instances (1 year)	Commit to 1 year in a specific region; significant upfront	~75% (partial upfront) to ~90% (all upfront)	Baseline (always running); CI/CD build ser
Reserved Instances (3 years)	3-year commitment for maximum discount	~56% (partial upfront) to ~90% (all upfront)	Baseline (all spare infra): NAT Gateway
Savings Plans (Compute)	Commit to a \$/hr spend amount, not specific instance type	~66% off spend up to \$50k/mo; up to 72% off	ShopFlow equipment: cost saving: \$5/hr Compute
Spot Instances	Bid on spare EC2 capacity; can be interrupted w/ 2-min notice	Up to 90% off on-demand	Batch processing, image resizing jobs, a

04-08

Auto Scaling for ECS Fargate — ShopFlow Configuration

ECS Fargate auto scaling works at the service level: ECS adds or removes tasks (containers) based on CloudWatch metrics. ShopFlow order-svc scales from 3 tasks at baseline to 30 tasks at Black Friday peak. Scaling is triggered by CPU utilization, memory, or custom metrics from Application Auto Scaling.

```
// ECS Fargate Auto Scaling in CDK
const orderSvc = new ecs.FargateService(this, "OrderSvc", {
  cluster,
  taskDefinition: orderTaskDef,
  desiredCount: 3, // start with 3 tasks
  vpcSubnets: {subnetType: ec2.SubnetType.PRIVATE_WITH_EGRESS},
  securityGroups: [ecsSg],
});
// Enable auto scaling
const scaling = orderSvc.autoScaleTaskCount({
  minCapacity: 3, // never go below 3 (1 per AZ for HA)
  maxCapacity: 30, // Black Friday ceiling
});
// Scale on CPU: add tasks when CPU > 60%; remove when < 60%
scaling.scaleOnCpuUtilization("CpuScaling", {
  targetUtilizationPercent: 60,
  scaleInCooldown: Duration.seconds(300), // wait 5 min before removing tasks
  scaleOutCooldown: Duration.seconds(60), // add tasks faster (scale out quickly)
});
// Scale on memory: add tasks when memory > 70%
scaling.scaleOnMemoryUtilization("MemoryScaling", {
  targetUtilizationPercent: 70,
});
```

■ Always set `scaleOutCooldown` lower than `scaleInCooldown`. You want to add capacity quickly when traffic spikes (60 seconds) but remove it slowly (300 seconds) to avoid oscillation — adding and removing tasks repeatedly as traffic fluctuates around the threshold.

04-09

EC2 Image Builder — Custom AMIs for ShopFlow

Rather than running a user data script on every new EC2 instance (slow, error-prone), ShopFlow uses EC2 Image Builder to create custom AMIs with Java 21, the CloudWatch agent, and security hardening pre-baked. New instances start 80% faster because setup is already done. Image Builder also runs CIS benchmarks and vulnerability scans on every image.

Step	Image Builder Component	ShopFlow Implementation
1. Start	Base image: Amazon Linux 2023 latest	Image Builder pulls the latest AL2023 AMI from AWS automatically
2. Build	Install software components	Install Java 21 (Corretto), CloudWatch agent, SSM agent, git, jq. Configure CIS Level 1
3. Test	Automated testing	Run: <code>java -version</code> (verify Java 21). <code>systemctl status amazon-cloudwatch-agent</code> (verify running)
4. Distribute	Push AMI to target regions and accounts	Share AMI to Dev, Staging, Prod accounts automatically. Notify CodePipeline to run integration tests
5. Scan	Security vulnerability scan	Inspector scans the AMI for CVEs. High/Critical CVEs block the build pipeline — forces remediation

04-10

EC2 Instance Lifecycle and State Management

Understanding the EC2 instance lifecycle prevents data loss and unexpected charges. An instance moves through states: `pending` → `running` → `stopping` → `stopped` → `terminated`. Each state has different billing implications and behaviors.

State	What Is Happening	Billing	Data Preserved?
Pending	Instance is starting; AWS is allocating hardware and installing the OS	No charge	N/A
Running	Instance is active and accessible	Charged per second for compute; EBS charges separately	Yes; EBS snapshots and instance store is active
Stopping	Instance is shutting down gracefully (OS shutdown); EBS is flushed	No compute charge; EBS still charged	Yes; EBS persists through stop/start
Stopped	Instance is off; hardware may be reallocated by AWS (private IP, EBS still charged)	No compute charge; EBS still charged	Yes; EBS persists, as Elastic SR are WIPED on hardware change

State	What Is Happening	Billing	Data Preserved?
Terminated	Instance permanently deleted; root EBS volume deleted (if DeleteOnTermination=true)	No charge (if DeletedOnTermination=true)	EBS deleted; additional EBS volume not deleted

■ Manage EC2 Instance State

1. EC2 → Instances → select your instance → Instance state dropdown
2. Start: resume a stopped instance (new public IP unless Elastic IP)
3. Stop: gracefully shut down (data preserved on EBS); good for saving costs on dev instances
4. Reboot: equivalent to OS restart; same public and private IP; EBS preserved
5. Terminate: permanent deletion; confirm the dialog (cannot undo)
6. Hibernate: save RAM contents to EBS root volume; on start, RAM is restored (fast resume)

04-11

Graviton3 Instances — Best Price-Performance for ShopFlow

AWS Graviton processors (ARM-based, designed by AWS) deliver 40% better price-performance than equivalent x86 Intel/AMD instances. ShopFlow targets Graviton3 (g-suffix: m6g, c6g, r6g) for all ECS Fargate tasks and EC2 instances. Java 21 on Graviton3 performs excellently — AWS has tested Spring Boot applications showing 30-40% cost reduction with no code changes.

Instance	Architecture	vCPU	RAM	Price/hr	vs x86 Equivalent
t3.medium	x86 (Intel)	2	4 GB	\$0.0416	Baseline
t4g.medium	Graviton2 (ARM)	2	4 GB	\$0.0336	19% cheaper, similar performance
m6i.large	x86 (Intel Ice Lake)	2	8 GB	\$0.096	Baseline
m6g.large	Graviton2 (ARM)	2	8 GB	\$0.077	20% cheaper, ~10% faster
c6i.2xlarge	x86 (Intel Ice Lake)	8	16 GB	\$0.34	Baseline
c6g.2xlarge	Graviton2 (ARM)	8	16 GB	\$0.272	20% cheaper, ~30% faster for Java
r6i.xlarge	x86 (Intel)	4	32 GB	\$0.252	Baseline
r6g.xlarge	Graviton2 (ARM)	4	32 GB	\$0.2016	20% cheaper, suitable for Redis hosts

■ Migrate ShopFlow ECS Fargate to Graviton3 on Day One

ECS Fargate supports ARM64 natively. Build your Docker images multi-arch: `docker buildx build --platform linux/amd64,linux/arm64`. In CDK, set `runtimePlatform: {cpuArchitecture: ecs.CpuArchitecture.ARM64}` on the task definition. This single change reduces your Fargate compute cost by 20% with no application code changes. At ShopFlow scale (\$8,000/month in Fargate), this saves \$1,600/month.

04-12

EC2 Placement Groups and High Availability

Placement groups control the physical location of EC2 instances in AWS data centers. The right placement group strategy affects: latency between instances (cluster placement = same rack = lowest latency), fault tolerance (spread placement = different racks = survive hardware failures), and throughput (partition placement = separate power/networking domains).

Placement Group Type	Physical Placement	Latency	Fault Tolerance	ShopFlow Use Case
Cluster	Same rack, same AZ	Lowest: ~25 μ s between instances in same rack	Lowest: single rack failure affects all	High Performance Computing (HPC) analytics
Spread	Different racks, different hardware	Normal	Highest: each instance on independent hardware	Critical services where each must survive in

Placement Group Type	Physical Placement	Latency	Fault Tolerance	ShopFlow Use Case
Partition	Groups of instances on separate hardware	power/network partitions	Availability with multiple AZs	Distributed databases (HDFS, Cassandra): k

```
// Spread placement group: each instance on different hardware (for HA)
const spreadGroup = new ec2.PlacementGroup(this, "SpreadGroup", {
  strategy: ec2.PlacementGroupStrategy.SPREAD,
});
// Attach to ASG by specifying placement group in launch template
```

04-13

Scheduled Scaling for Black Friday

Scheduled scaling lets you proactively add capacity before you need it, rather than waiting for CPU to spike (which takes minutes to detect and respond to). ShopFlow pre-scales ECS tasks at 8pm Thursday for Black Friday, ensuring capacity is ready when traffic hits at midnight.

```
// Scheduled scaling in CDK – Application Auto Scaling
import * as appscaling from "aws-cdk-lib/aws-applicationautoscaling";
const scalingTarget = new appscaling.ScalableTarget(this, "OrderSvcScaling", {
  serviceNamespace: appscaling.ServiceNamespace.ECS,
  resourceId: "service/shopflow-cluster/order-svc",
  scalableDimension: "ecs:service:DesiredCount",
  minCapacity: 3,
  maxCapacity: 30,
});
// Pre-scale at 8pm Thursday (UTC) = midnight Black Friday IST
scalingTarget.scaleOnSchedule("BlackFridayScaleOut", {
  schedule: appscaling.Schedule.cron({hour:"20", minute:"0", weekday:"THU"}),
  minCapacity: 20, // jump from 3 to 20 tasks proactively
  maxCapacity: 40, // allow auto scaling up to 40 if needed
});
// Scale back down at 9am Friday
scalingTarget.scaleOnSchedule("BlackFridayScaleIn", {
  schedule: appscaling.Schedule.cron({hour:"9", minute:"0", weekday:"FRI"}),
  minCapacity: 3,
  maxCapacity: 30,
});
```

04-14

Monitoring EC2 with CloudWatch

CloudWatch provides built-in metrics for EC2 instances: CPUUtilization, NetworkIn/Out, DiskReadOps, StatusCheckFailed. For memory utilization and disk usage, you must install the CloudWatch agent — EC2 does not expose these by default because AWS does not have access to the OS internals.

EC2 CloudWatch Metrics — shopflow-bastion-host

■ Available metrics (built-in — no agent required)

- CPUUtilization — CPU usage % — 5-minute default; switch to 1-minute for \$0.30/metric/month
- NetworkIn — Bytes received by the instance per period
- NetworkOut — Bytes sent by the instance per period
- DiskReadBytes / DiskWriteBytes — EBS throughput
- StatusCheckFailed — 1 if EC2 hardware or OS check fails; triggers Auto Scaling replacement

■ Requires CloudWatch Agent (install on instance)

- mem_used_percent — RAM utilization: 78% (CRITICAL: not available without agent)
- disk_used_percent — Disk usage: 45% (not available without agent)
- swap_used_percent — Swap usage: 0% (not available without agent)
- Install agent: `sudo yum install amazon-cloudwatch-agent -y` or via EC2 Image Builder

04-15

Hands-On Lab — Full EC2 + Auto Scaling Setup

This end-to-end lab builds a simple web tier with auto scaling. You will launch an EC2 instance, install a simple web server, create an AMI from it, build an Auto Scaling Group, attach an ALB, and watch it scale.

Step	Action	Expected Result
1. Launch EC2	t3.micro, Amazon Linux 2023, shopflow-vpc public subnet, shopflow-bastion-sg, status checks pass, public IP assigned	Instance running, status checks pass, public IP assigned
2. Connect EC2 SSM	EC2 → SSM → Connect → Session Manager	Browser terminal opens; run: <code>whoami</code> (shows <code>ssm-user</code>)
3. Install web server	<code>sudo yum install -y httpd; sudo systemctl start httpd; sudo systemctl enable httpd</code>	<code>curl http://localhost</code> returns Apache test page
4. Create AMI	EC2 → Actions → Image and templates → Create image → name: shopflow-web-ami	AMI created, web-ami-xxxxxxx
5. Create Launch Template	EC2 → Launch Templates → Create → step 4; t3.micro; shopflow-ecs-sg; no public IP	Launch template version 1 created
6. Create ALB	EC2 → Load Balancers → Application → shopflow-lab-alb → public subnets; target group: shopflow-lab-tg	ALB created, web-01-xxxxxx/
7. Create ASG	EC2 → Auto Scaling → Groups → shopflow-lab-asg → t3.micro; Desired 2, Max 6; private subnets; attach to ALB target group	ASG created, 2 instances launching with ALB; health checks pass
8. Test scaling	<code>stress --cpu 4</code> (install stress: <code>sudo dnf install stress -y</code>) → CloudWatch alarm triggers; 3rd instance launches after ~2 min	CloudWatch alarm triggers; 3rd instance launches after ~2 min
9. Verify	EC2 → Auto Scaling Groups → Activity tab	Scale-out event shown with timestamp and new instance ID

04-16

Spot Instance Interruption Handling in Java

Spot instances save 70-90% but can be reclaimed by AWS with a 2-minute warning. For ShopFlow batch analytics jobs (which process historical order data), Spot interruptions are acceptable — the job checkpoints to S3 every 30 seconds and restarts from the last checkpoint. Here is the pattern.

```
// Java 21 - Spot interruption detection (runs every 5 seconds)
@Scheduled(fixedDelay = 5000)
public void checkSpotInterruption() {
    try {
        // AWS makes termination notice available at this endpoint
        HttpClient client = HttpClient.newHttpClient();
```

```
HttpRequest req = HttpRequest.newBuilder()
    .uri(URI.create("http://169.254.169.254/latest/meta-data/spot/termination-time"))
    .build();
HttpResponse resp = client.send(req, BodyHandlers.ofString());
if (resp.statusCode() == 200) {
    // Termination notice received – checkpoint immediately
    log.warn("Spot interruption at: {}", resp.body());
    checkpointService.saveProgressToS3(); // save current offset
    gracefulShutdown(); // drain queue, close DB connections
}
// If 404: no interruption notice; continue processing
} catch (Exception e) {
    // Network error checking IMDS; not an interruption – continue
}
}
```

■ AWS Spot best practices: use Spot Fleet or ASG with Spot instances across 3+ instance types (e.g., c6g.2xlarge, c6i.2xlarge, m6g.2xlarge) to reduce interruption probability — when one type has no capacity, ASG falls back to another type automatically.

04-17

EC2 vs ECS Fargate — When to Use Which

The most common question for ShopFlow developers: "Should I use EC2 directly or ECS Fargate?" Here is the decision framework. ShopFlow uses Fargate for all microservices but EC2 for specific operational needs.

Scenario	Use EC2	Use ECS Fargate
Long-running microservice (order-svc, product-catalog)	No — you manage OS patching, capacity, VM storage, etc.	Yes — AWS manages all the compute plane; you manage only container images
Batch job that needs specialized hardware (GPU, Fargate ML Mac)	Yes — GPU, Fargate ML Mac GPU support; c5d (local NVMe) instances for local NVMe storage	No — EC2 (local NVMe) instances needed
Database administration / bastion host	Yes — small t3.micro; occasional use; SSM Session Manager, EC2 instance is simpler	No — SSM Session Manager, EC2 instance is simpler
CI/CD build jobs (CodeBuild alternative)	Possible — EC2-based CodeBuild	Yes — ECS tasks as on-demand build runners (only pay when built)
Machine learning training (PyTorch, TensorFlow)	Yes — GPU instances (p3, p4d); Spot instances for 70-80% savings	No — Fargate has no GPU support
Custom OS requirements (specialized kernel, kernel modules)	Yes, kernel controls OS entirely	No — Fargate OS is managed by AWS; cannot install kernel modules

04-18

Auto Scaling Lifecycle Hooks

Lifecycle hooks give you control over what happens when an instance launches or terminates. When an instance launches, you might want to: warm the application cache before the instance joins the target group. When an instance terminates, you want to: drain in-flight requests and deregister from service discovery. Lifecycle hooks pause the scaling action for up to 2 hours to complete custom actions.

Hook Type	When It Fires	ShopFlow Use Case	Action
autoscaling:EC2_INSTANCE_LAUNCHING	Instance just launched; not yet in InstanceProfile	Service application cache: pre-load top 100 products	Call LambdaWarmupLambda
autoscaling:EC2_INSTANCE_TERMINATING	ScalingGroup decided to terminate instance	Drain in-flight requests: wait for all open HTTP connections	Call drainLambda

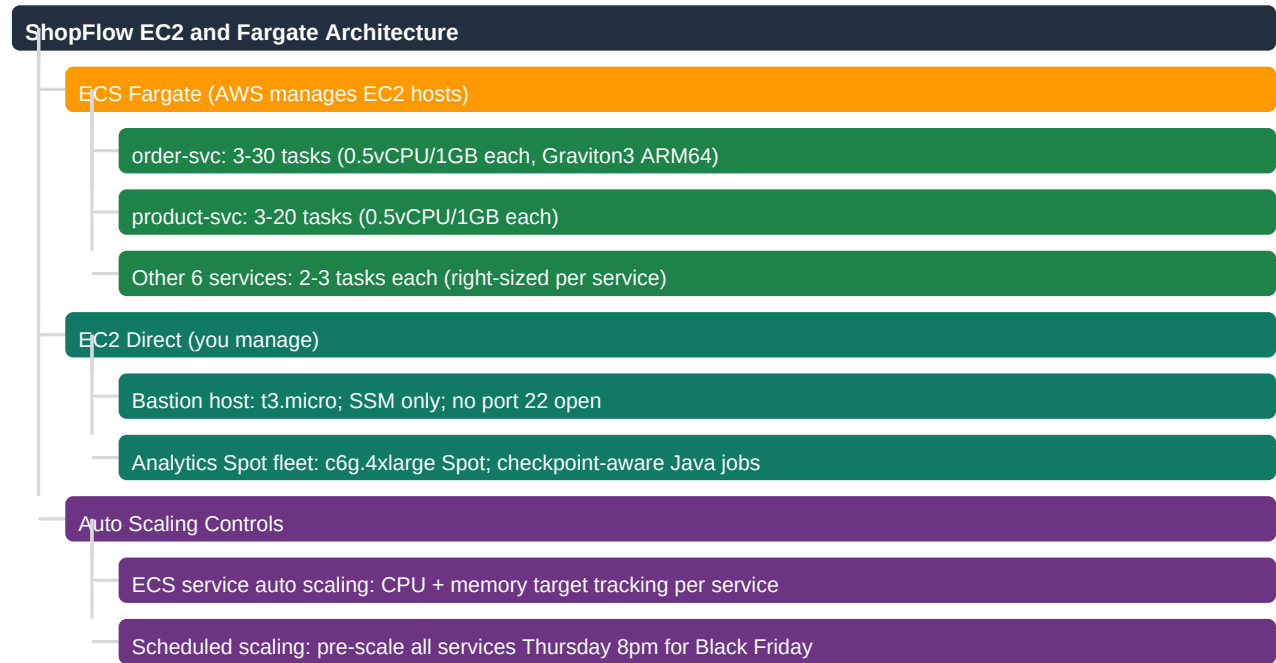
```
// Lifecycle hook CDK
asg.addLifecycleHook("LaunchHook", {
    lifecycleTransition: autoscaling.LifecycleTransition.INSTANCE_LAUNCHING,
    heartbeatTimeout: Duration.minutes(5), // max time for warmup
    notificationTarget: new LambdaHook(warmupLambda),
    defaultResult: autoscaling.DefaultResult.CONTINUE,
});
```

04-19

EC2 Architecture Summary — ShopFlow Complete

Here is the complete picture of how EC2 and auto scaling serve ShopFlow. Fargate handles all microservices; EC2 appears only where Fargate cannot.

Component	Instance Type	Count	Scaling Strategy	Monthly Cost (est.)
ECS Fargate — order-svc	0.5 vCPU / 1 GB RAM (Fargate Graviton)	30 Black Friday	CPU 60% target tracking + scheduled Black Friday scaling	\$45 baseline / \$50 peak
ECS Fargate — product-svc	0.5 vCPU / 1 GB RAM	3 baseline → 20 peak	CPU + memory target tracking	\$45 baseline / \$300 peak
ECS Fargate — other 6 services	0.25-1 vCPU / 0.5-2 GB	2-3 each baseline	Individual CPU scaling per service	~\$150 total baseline
Bastion host (admin access)	t3.micro (EC2)	1 (always on)	None (manual management)	\$8/month
Analytics batch jobs	c6g.4xlarge Spot (EC2)	0-10 (on-demand jobs)	Scheduled jobs; Spot interruption-aware	\$0 (idle) / ~\$20 per 4-h batch
CodeBuild runners	Managed by CodeBuild (EC2 on-premise)	10 (per build)	Auto-provisioned by CodeBuild; no management	\$30/month (10 builds)



04-20 EC2 Networking Deep Dive — ENI, ENA, and Bandwidth

Each EC2 instance type has a maximum network bandwidth that determines how fast data can flow in and out. For ShopFlow, network bandwidth matters for: ECS tasks pulling Docker images from ECR (large images can be 1-2 GB), Aurora reading/writing large result sets, and product image uploads to S3. Understanding bandwidth limits helps you right-size instances.

Instance	Network Bandwidth	Baseline vs Burst	ShopFlow Impact
t3.medium (2vCPU/4GB)	Up to 5 Gbps (burst)	Baseline not specified; bursts to 5 Gbps briefly	Only for bastion host; insufficient for high-throughput
m6g.large (2vCPU/8GB)	Up to 10 Gbps	12.5 Gbps baseline; burst to higher	Good for general ECS service tasks
c6g.2xlarge (8vCPU/16GB)	Up to 10 Gbps	10 Gbps baseline	Good for CPU-intensive analytics batch jobs
c6gn.large	Up to 25 Gbps	25 Gbps baseline (n = network optimized)	Needed if Kinesis/streaming tasks process v
c6g.4xlarge (16vCPU/32GB)	Up to 12 Gbps	12 Gbps baseline	Sufficient for most ShopFlow analytics batch

■ For ECS Fargate, the network bandwidth is determined by the vCPU allocation: 0.25 vCPU = 2.5 Gbps, 4 vCPU = 10 Gbps. If order-svc is doing large DynamoDB reads, increase the Fargate vCPU allocation to get more network bandwidth, not just more compute.

AWS Compute Optimizer analyzes actual CPU and memory usage of your EC2 instances and ECS Fargate tasks over 14 days and recommends right-sized alternatives. For ShopFlow, Compute Optimizer often finds that ECS tasks are allocated 2x more CPU and RAM than they actually use, identifying significant savings.

■ Use Compute Optimizer to Right-Size ShopFlow

1. Search "Compute Optimizer" in the console → Opt in (click "Get started" → opt in for your account)
2. Wait 24-48 hours for Compute Optimizer to analyze your workloads
3. Go to Compute Optimizer → EC2 instances → review findings
4. For each instance: see Current vs Recommended (e.g., m6i.large → t3.large: \$18/month savings)
5. Go to ECS on Fargate → review task recommendations
6. Example: order-svc allocated 1 vCPU/2GB; actual usage: 0.3 vCPU/800MB → recommend 0.5 vCPU/1GB (50% cost reduction)
7. Click "View recommendation details" → see CPU and memory utilization charts over 14 days
8. Apply recommendation: update CDK task definition cpu: 512, memoryLimitMiB: 1024; deploy
9. Monitor for 48 hours; recheck Compute Optimizer to confirm savings maintained

ShopFlow Service	Current Allocation	Actual Usage (avg)	Recommended	Monthly Saving
order-svc (x3 tasks)	1 vCPU / 2 GB	0.3 vCPU / 900 MB	0.5 vCPU / 1 GB	\$27/month (3 tasks)
product-svc (x3 tasks)	1 vCPU / 2 GB	0.2 vCPU / 600 MB	0.25 vCPU / 1 GB	\$32/month (3 tasks)
search-svc (x2 tasks)	2 vCPU / 4 GB	1.2 vCPU / 2.1 GB	1 vCPU / 2 GB	\$14/month (2 tasks)
notification-svc (x2 tasks)	0.5 vCPU / 1 GB	0.1 vCPU / 300 MB	0.25 vCPU / 0.5 GB	\$8/month (2 tasks)
Total savings from right-sizing				~\$81/month (\$972/year)

ShopFlow maintains a "golden AMI" — a base AMI with all required software, security hardening, and configuration baked in. Every new EC2 instance launches from this AMI. The pipeline runs weekly, pulling the latest Amazon Linux 2023 base, applying security patches, running CIS hardening, and scanning for CVEs before making the AMI available to Auto Scaling Groups.

Golden AMI Pipeline — Weekly Schedule

Step 1: EC2 Image Builder pulls latest Amazon Linux 2023 base AMI

Step 2: Build phase — install software components

Install Java 21 Corretto + Spring Boot 3 dependencies

Install CloudWatch Agent + SSM Agent + AWS CLI v2

Apply CIS Level 1 hardening (disable root SSH, auditd, firewalld)

Step 3: Test phase — automated validation

java -version: must return "21.x"

CIS benchmark scan: must pass 90%+ controls

Inspector CVE scan: zero Critical or High CVEs (block pipeline if found)

Step 4: Distribute — share AMI to all ShopFlow accounts

Share to: Dev, Staging, Prod, DR accounts

Tag AMI: shopflow:golden=true; shopflow:week=2024-W48

Step 5: Update Launch Templates to point to new AMI version

CDK: update launch template AMI ID; deploy; ASG refreshes instances

04-BP

Best Practices and Anti-Patterns

✓ Use SSM Session Manager instead of SSH for all EC2 access — eliminates the attack surface of exposed port 22, removes the burden of managing SSH keys, and provides automatic session audit logging

✓ Always encrypt EBS volumes — enable encryption at rest for all EC2 volumes; use AWS-managed keys or customer-managed KMS keys for sensitive workloads

✓ Use Compute Savings Plans over Reserved Instances when possible — Savings Plans apply automatically to any EC2, Fargate, or Lambda usage within the committed spend, without locking to specific instance types or regions

✓ Set separate scaleOut and scaleIn cooldown periods — scale out aggressively (60 seconds) to handle traffic spikes quickly; scale in conservatively (5-10 minutes) to avoid removing capacity prematurely

✓ Use Launch Templates with versioning, not Launch Configurations — Launch Templates support versioning (roll back to previous config in 30 seconds), multiple instance types for Spot diversification, and T3/T4g unlimited burst credit mode

✓ Bake AMIs with Image Builder rather than using long user data scripts — image baking reduces instance boot time from 5-10 minutes to 60 seconds, makes configuration testable, and gives you a scannable artifact for security compliance

■ Do not give EC2 instances an IAM role with AdministratorAccess — the EC2 instance profile should have only the minimum permissions needed; if the instance is compromised, the attacker gets those permissions

■ Never store sensitive data on instance store volumes — instance store is ephemeral and is lost when the instance stops; use EBS or S3 for persistence

■ Do not use t3/t4g instances for production database workloads — burstable instances throttle CPU when CPU credits run out; use m6g or r6g families for consistent database performance

04-QR

Quick Reference and Interview Q&A

Question	Answer
What is the difference between stopping the instance and EC2 instance?	Stopping the instance and EC2 instance, EBS root volume is preserved; when you start it again, the s
What happens when a Spot Instance is terminated?	Amazon gives a 2-minute warning via instance metadata (GET http://169.254.169.254/latest/meta-data/s
How does EC2 Auto Scaling decide when to scale up or down?	Target Scaling ASG calculates the metric (e.g., CPU) across all instances, compares to target (60%
What is the difference between an AMI and a Snapshot?	An AMI (Amazon Machine Image) is a complete machine image used to launch EC2 instances — it in